

strategies, enabling organisations to fortify their defences, pre-emptively address vulnerabilities, and uphold the resilience of their digital ecosystems in the face of evolving threats and adversaries.

Despite the critical importance of network and port scanning in cybersecurity, existing tools often lack efficiency, accuracy, and user-friendliness. Current solutions may struggle to provide comprehensive insights into network topology and port configurations, hindering effective security assessments and vulnerability analyses. Addressing these limitations, let's develop a sophisticated network and port scanning tool that overcomes existing challenges, enabling cybersecurity professionals to identify active devices and open ports swiftly and accurately, thereby enhancing network security and resilience against evolving cyber threats.

Getting started

Here are the steps that will help us to get started.

- Install scapy library.
- Import the necessary modules — socket, scapy, thread, lock.
- Define function: `scanHost(ip, startPort, endPort)` – Scan TCP ports on a single host within a specified range.
- Define function: `scanRange(ip, startPort, endPort)` – Scan TCP ports on all hosts within a network.
- Define function: `tcpScan(ip, startPort, endPort)` – Scan TCP ports on a specific IP address.
- Define function: `network_scan(target_ip)` – Scan devices in a network using ARP.
- Use socket to create TCP connections to each port within the specified range.
- Print open ports if the connection is successful.
- Use scapy to create ARP packets and send them to discover devices in the network.

- Extract IP and MAC addresses from the responses.
- Print the discovered devices.
- Test the script with various inputs to ensure it behaves as expected.
- Optimise the script for efficiency and usability.

To begin with we would need Python version 3.x and any Python-compatible IDE (e.g., PyCharm, VSCode). A Python-based network and port scanner uses modules like scapy and socket to identify active hosts and open ports within a network.

Next, load the following libraries:

- *library(scapy)*: This is used for interacting with the packets on the network.
- *library(socket)*: This establishes network connections over various network protocols such as TCP or UDP to send and receive data.

Defining function `scanHost()` and `scanRange()`

The code initialises a print lock `'print_lock = Lock()'` to synchronise the console output. This is utilised in critical sections to ensure orderly printing when multiple threads are involved, and prevents potential issues with concurrent printing during TCP port scanning.

scanHost(ip, startPort, endPort):

This function initiates a TCP port scan on a single host identified by the IP parameter. It iterates through the range of ports from `startPort` to `endPort` and calls the `tcp_scan` function to perform the scan. It prints messages indicating the start and completion of the scan for that host.

scanRange(network, startPort, endPort): This function conducts TCP port scanning on a range of hosts within a network. It iterates through host addresses in the network range (e.g.,

```
1  import socket
2  from scapy.layers.l2 import ARP, Ether
3  from scapy.sendrecv import srp
4
5  from threading import Thread, Lock
6
```

Figure 1: Installing libraries and importing packages

```
8  print_lock = Lock()
9
10 def scanHost(ip, startPort, endPort):
11     print('[*] Starting TCP port scan on host %s' % ip)
12
13     tcp_scan(ip, startPort, endPort)
14     print('[+] TCP scan on host %s complete' % ip)
15
16 def scanRange(network, startPort, endPort):
17     print('[*] Starting TCP port scan on network %s.0' % network)
18     for host in range(1, 255):
19         ip = network + '.' + str(host)
20         tcp_scan(ip, startPort, endPort)
21
22     print('[+] TCP scan on network %s.0 complete' % network)
23
```

Figure 2: Defining function `scanHost()` and `scanRange()`